



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN
HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 9-2

NOMBRE COMPLETO: BRANDON HERNANDEZ SOLIS

N° de Cuenta: 318263113

GRUPO DE LABORATORIO: 2

GRUPO DE TEORÍA: 6

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 27/10/2024

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEHC MONOPOLY. animado desplazándose las letras de izquierda a derecha como si fuera letrero LCD/LED de forma cíclica

Para la actividad se requirió crear primero la textura dentro de GIMP donde se tuviera la oración, luego se le dio transparencia y se exporto como tga para poder agregarla sin problemas a la carpeta de texturas.

Después se requirió importar de nuevo el modelo del arco de meta, así como las animaciones que se le realizaron en la práctica pasada.

Luego fue necesario crear una nueva malla para el letrero, donde las coordenadas UV estén ajustadas a nuestra textura, después de eso se renderizo la malla justo después del modelo del arco, se debe respetar la jerarquía por lo que la malla va ligada al modelo animado del letrero.

Después solo se hicieron ajustes mínimos para colocar adecuadamente la malla sobre el letrero animado y se reutilizo la sección de código de los números que van desplazándose.

Main.cpp:

```
80     Model Dragon_cabeza_no;
81
82     Texture LetreroTexture;
83
84     Model Letrero;
85     Model Arco;
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```

299
300 //Malla ajustada
301 unsigned int letreroIndices[] = {
302     0, 1, 2,
303     0, 2, 3,
304 };
305
306 GLfloat letreroVertices[] = {
307     -0.5f, 0.0f, 0.5f,    0.0f, 1.0f,    0.0f, 1.0f, 0.0f, //Izquierda Arriba
308     0.5f, 0.0f, 0.5f,    1.0f, 1.0f,    0.0f, 1.0f, 0.0f, //Derecha Arriba
309     0.5f, 0.0f, -0.5f,   1.0f, 0.0f,    0.0f, 1.0f, 0.0f, //Derecha Abajo
310     -0.5f, 0.0f, -0.5f,  0.0f, 0.0f,    0.0f, 1.0f, 0.0f, //Izquierda Abajo
311
312 };

```

```

342
343
344 //Nueva malla
345 Mesh* obj8 = new Mesh();
346 obj8->CreateMesh(letreroVertices, letreroIndices, 32, 6);
347 meshList.push_back(obj8);
348
349
350

```

```

390
391 LetreroTexture = Texture("Textures/letrero.tga");
392 LetreroTexture.LoadTextureA();
393

```

```

421
422 //Arco Letrero
423 Letrero = Model();
424 Letrero.LoadModel("Models/Letrero.obj");
425 Arco = Model();
426 Arco.LoadModel("Models/Estructura.obj");
427

```

```

499
500 //Letrero
501
502 movLetrero = 0.0f;
503 offsetLetrero = 0.06f; //Velocidad Movimiento
504 rotLetrero = 0.0f;
505 offsetRotLetrero = 3.0f; //Velocidad Rotacion
506
507 estadoArco = 0;
508

```

```

975 // Ejercicio Letrero
976
977 // Renderizado de Modelos
978
979 model = glm::mat4(1.0);
980 model = glm::translate(model, glm::vec3(-10.0f, -1.0f, -3.0f));
981 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
982 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
983 Arco.RenderModel();
984
985 model = glm::mat4(1.0);
986 model = glm::translate(model, glm::vec3(-8.9f, movLetrero + 6.5f, 2.0f));
987 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
988 model = glm::rotate(model, rotLetrero * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
989 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
990 Letrero.RenderModel();
991
992 // Animacion
993
994 if (glfwGetTime() > 3) {
995
996     if (estadoArco == 0) {
997         if (movLetrero > -7.0f) {
998             movLetrero -= offsetLetrero * deltaTime;
999         }
1000     }
1001     else {
1002         estadoArco = 1;
1003     }
1004
1005     if (estadoArco == 1) {
1006         if (rotLetrero <= 360.0f) {
1007             rotLetrero += offsetRotLetrero * deltaTime;
1008         }
1009     }
1010     else {
1011         rotLetrero = 0;
1012         estadoArco = 2;
1013     }
1014 }

```

```

1036
1037 // Incremento del desplazamiento horizontal
1038 //toffsetnumerocambiau += 0.001; //Aumentar el offset 165 HZ
1039 toffsetnumerocambiau += 0.01; //Reducir el offset 60 HZ
1040
1041 // Reset en el ciclo de la textura
1042 if (toffsetnumerocambiau > 1.0)
1043     toffsetnumerocambiau = 0.0;
1044 toffsetnumerov = 0.0;
1045 toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);
1046
1047 // Transformaciones del letrero
1048 model = glm::translate(model, glm::vec3(0.1f, 0.0f, 0.0f));
1049 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1050 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
1051 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
1052 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1053
1054 // Aplicación de la textura y material
1055 glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
1056 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1057 color = glm::vec3(1.0f, 1.0f, 1.0f);
1058 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1059 LetreroTexture.UseTexture();
1060 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
1061 meshList[7]->RenderMesh();

```

Resultados:



2. Separar las cabezas del Dragón y agregar las siguientes animaciones:

- Movimiento del cuerpo ida y vuelta.
- Aleteo
- Cada cabeza se mueve de forma diferente de acuerdo con una función/algoritmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc..)
- Cada cabeza debe verse de un color diferente: roja, azul, verde, blanco, café.

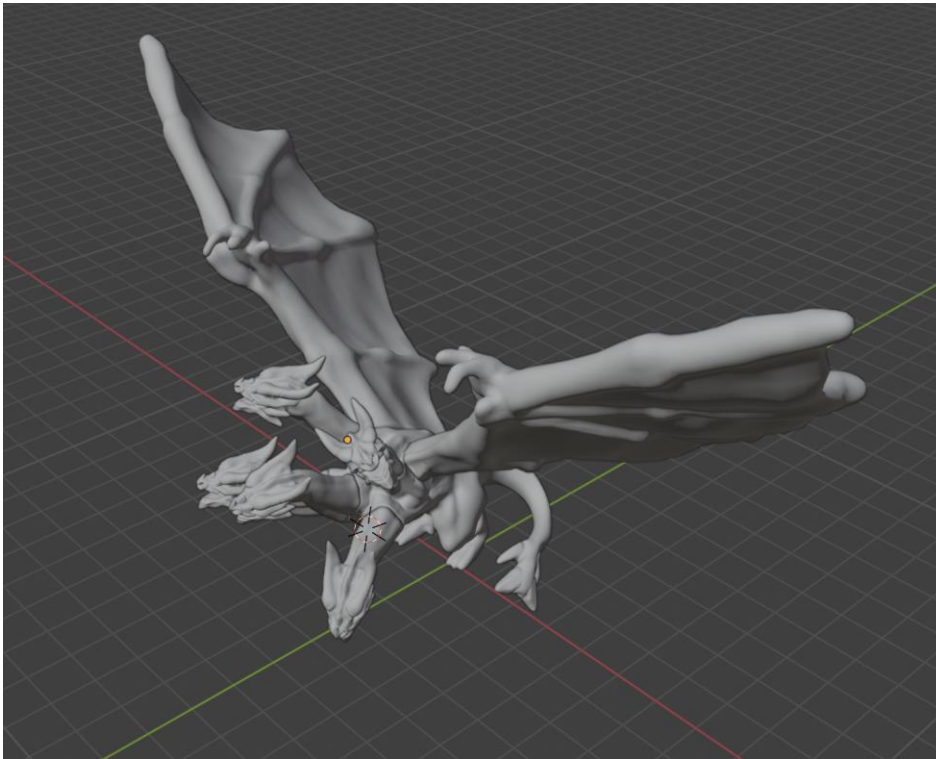
Esta actividad fue laboriosa debido a la gran cantidad de pasos para completarla. Empezamos con la división en secciones del modelo del dragón en Blender, después se llenan los huecos generados y se asigna el pivote y se coloca el en origen para cada modelo. Después se les dio color a las cabezas usando una textura de una paleta de colores y por último se exportan los modelos individuales.

Lo siguiente es importar los modelos al código y organizar las jerarquías de los modelos para que las animaciones se puedan realizar adecuadamente, esto se realizó simplemente con translate y se armó el modelo del dragón de nuevo.

Las animaciones se realizaron usando secciones de código utilizado en el ejercicio de clase, pero para la animación de cada cabeza se definió una función la cual retornaba los valores necesarios de movimiento para cada animación. Se usaron 5 animaciones diferentes las cuales afectada a los modelos de las cabezas usando rotate.

El desplazamiento del modelo se realizó similar a la animación del carro que recorría el tablero, en esta ocasión se añadió una función para hacer el giro de 180 grados de manera realista, por lo que se utilizaban rotate y translate para conseguir este efecto en el desplazamiento.

Blender:



 Colecciones de la escena

- ▼  Cuerpo ☒  
 - >  Base   
 - >  Dragon_Cuerpo   
- ▼  Cabezas ☒  
 - >  Dragon_Cabeza_C   
 - >  Dragon_Cabeza_LD   
 - >  Dragon_Cabeza_LU   
 - >  Dragon_Cabeza_RD   
 - >   Dragon_Cabeza_RU   
- ▼  Alas ☒  
 - >  Ala_Derecha   
 - >  Ala_Izquierda   

Main.cpp:

```
44 //variables para animación
45 float movCoche;
46 float movOffset;
47 float rotllanta;
48 float rotllantaOffset;
49 bool avanza;
50 float toffsetflechau = 0.0f;
51 float toffsetflechav = 0.0f;
52 float toffsetnumerou = 0.0f;
53 float toffsetnumerov = 0.0f;
54 float toffsetnumerocambiau = 0.0;
55 float angulovaria = 0.0f;
```

```
57 //Variables nuevas
58
59 bool alternar;
60 bool rKeyPressed;
61
62 float movDragon;
63 float movDragonOffset;
64 int estado;
```

```
72
73 Model DragonAlaIzq;
74 Model DragonAlaDer;
75
76 Model Dragon_Cabeza_C;
77 Model Dragon_Cabeza_LD;
78 Model Dragon_Cabeza_LU;
79 Model Dragon_Cabeza_RD;
80 Model Dragon_Cabeza_RU;
```

```

87 // Animacion para giro
88 float anguloRotacion;
89 float traslacionX;
90 float traslacionZ;
91 float anguloObjetivo ;
92
93 float velocidadRotacion;
94 float distanciaTraslacion;
95

```

```

97 // Funciones de animacion
98
99 v float RotacionEspiralArquimedes(float deltaTime) {
100     float a = 2.0f;
101     float b = 1.5f;
102     return (a + b * sin(deltaTime * 0.1f)) * toRadians;
103 }
104
105 v float RotacionSenoidal(float deltaTime) {
106     float amplitud = 10.0f;
107     float frequency = 0.1f;
108     return (amplitud * sin(frequency * deltaTime)) * toRadians;
109 }
110
111 v float RotacionLemniscata(float deltaTime) {
112     float a = 25.0f;
113     return (a * sin(deltaTime) * cos(deltaTime)) * toRadians;
114 }
115
116 v float RotacionCircularPulsante(float deltaTime) {
117     float baseRotation = 5.0f;
118     float pulsation = sin(deltaTime * 0.5f);
119     return (baseRotation + pulsation * 10.0f) * toRadians;
120 }
121
122 v float RotacionOscilatoriaDiagonal(float deltaTime) {
123     float amplitud = 20.0f;
124     float frequency = 0.05f;
125     return (amplitud * sin(frequency * deltaTime) * cos(frequency * deltaTime)) * toRadians;
126 }
127
128

```

```

401 // Modelos de la practica
402
403
404 Dragon_M = Model();
405 Dragon_M.LoadModel("Models/Dragon.obj");
406 DragonAlaIzq = Model();
407 DragonAlaIzq.LoadModel("Models/DragonAlaIzq.obj");
408 DragonAlaDer = Model();
409 DragonAlaDer.LoadModel("Models/DragonAlaDer.obj");
410
411 Dragon_Cabeza_C = Model();
412 Dragon_Cabeza_C.LoadModel("Models/Dragon_Cabeza_C.obj");
413 Dragon_Cabeza_LD = Model();
414 Dragon_Cabeza_LD.LoadModel("Models/Dragon_Cabeza_LD.obj");
415 Dragon_Cabeza_LU = Model();
416 Dragon_Cabeza_LU.LoadModel("Models/Dragon_Cabeza_LU.obj");
417 Dragon_Cabeza_RD = Model();
418 Dragon_Cabeza_RD.LoadModel("Models/Dragon_Cabeza_RD.obj");
419 Dragon_Cabeza_RU = Model();
420 Dragon_Cabeza_RU.LoadModel("Models/Dragon_Cabeza_RU.obj");
421

```

```

490
491     estado = 1;
492     movDragon = 0.0f;
493     movDragonOffset = 0.1f;
494
495     velocidadRotacion = 0.5f; // Velocidad de rotación en grados por segundo
496     distanciaTraslacion = 30.0f; // Distancia total a recorrer en X y Z
497     traslacionX = 0.0f;
498     traslacionZ = 0.0f;
499
533
534     if (glfwGetTime() > 1) {
535
536         if (estado == 1) {
537             if (movDragon > -50.0f) {
538                 movDragon -= movDragonOffset * deltaTime;
539             }
540             else {
541                 anguloObjetivo = 180; // Angulo del siguiente estado
542                 anguloRotacion = 0;
543                 estado = 2;
544             }
545         }
546
547         if (estado == 2) {
548
549             if (anguloRotacion < anguloObjetivo) {
550                 anguloRotacion += velocidadRotacion * deltaTime;
551
552                 // Calcula la traslación en X y Z para moverse en arco
553                 traslacionZ = distanciaTraslacion * sin(glm::radians(anguloRotacion));
554                 traslacionX = distanciaTraslacion * (1 - cos(glm::radians(anguloRotacion)));
555             }
556             else {
557                 estado = 3;
558                 // Limita el ángulo
559                 anguloRotacion = anguloObjetivo;
560             }
561         }
562     }
563
564 //Ejercicio Dragon
565
566 model = glm::mat4(1.0);
567 modelaux = glm::mat4(1.0);
568
569 model = glm::translate(model, glm::vec3(0.0f + movDragon, 0.0f + 0.8*sin(glm::radians(angulovaria * 5)), 6.0f));
570 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
571 // Se rota 90 grados en Y para la direccion
572 model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
573
574 //Animacion giro
575 // Aplica la traslación en X y Z
576 model = glm::translate(model, glm::vec3(traslacionX, 0.0f, traslacionZ));
577 model = glm::rotate(model, glm::radians(anguloRotacion), glm::vec3(0.0f, 1.0f, 0.0f));
578
579 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
580 /*color = glm::vec3(0.0f, 1.0f, 0.0f);
581 glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
582 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
583 Dragon_M.RenderModel();
584
585 // Alas
586 model = glm::translate(model, glm::vec3(-0.7f, 3.4f, 0.7f));
587 model = glm::rotate(model, (32.0f * sin(glm::radians(angulovaria * 5))) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
588 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
589 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
590 DragonAlaDer.RenderModel();
591
592 model = modelaux;
593
594 model = glm::translate(model, glm::vec3(0.3f, 3.4f, 0.8f));
595 model = glm::rotate(model, (32.0f * sin(glm::radians(angulovaria * 5))) * toRadians, glm::vec3(0.0f, 0.0f, -1.0f));
596 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
597 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
598 DragonAlaIzq.RenderModel();
599
600

```

```

738 // Cabezas
739
740
741 model = modelaux;
742
743 // Roja
744 model = glm::translate(model, glm::vec3(-0.2f, 3.0f, 2.6f));
745 model = glm::rotate(model, RotacionEspiralarquimedes(angulovaria), glm::vec3(1.0f, 0.0f, 0.0f)); // Giro en el eje X
746 model = glm::rotate(model, RotacionEspiralarquimedes(angulovaria * 0.3), glm::vec3(0.0f, 1.0f, 0.0f)); // Giro en el eje Y
747
748 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
749 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
750 Dragon_Cabeza_C.RenderModel();
751
752 model = modelaux;
753
754 // Verde
755 model = glm::translate(model, glm::vec3(0.4f, 3.8f, 2.2f));
756 model = glm::rotate(model, RotacionLemniscata(angulovaria * 0.05), glm::vec3(1.0f, 0.0f, 0.0f)); // Giro en el eje X
757 model = glm::rotate(model, RotacionLemniscata(angulovaria * 0.02), glm::vec3(0.0f, 0.0f, 1.0f)); // Giro en el eje Y
758
759 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
760 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
761 Dragon_Cabeza_LU.RenderModel();
762
763 model = modelaux;
764
765 // Blanca
766 model = glm::translate(model, glm::vec3(-1.0f, 2.7f, 2.5f));
767
768 model = glm::rotate(model, RotacionCircularPulsante(angulovaria*0.1), glm::vec3(1.0f, 0.0f, 0.0f));
769 model = glm::rotate(model, RotacionCircularPulsante(angulovaria*0.2), glm::vec3(0.0f, 0.0f, 1.0f));
770
771 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
772 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
773 Dragon_Cabeza_RD.RenderModel();
774
775 model = modelaux;
776
777 // Azul
778 model = glm::translate(model, glm::vec3(0.6f, 2.7f, 2.5f));
779 model = glm::rotate(model, RotacionSenoidal(angulovaria), glm::vec3(0.0f, 0.0f, 1.0f)); // Giro en el eje Z
780
781 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
782 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
783 Dragon_Cabeza_LD.RenderModel();
784
785 model = modelaux;
786
787 // Cafe
788 model = glm::translate(model, glm::vec3(-0.8f, 3.8f, 2.1f));
789 model = glm::rotate(model, RotacionOscilatoriaDiagonal(angulovaria), glm::vec3(1.0f, 1.0f, 0.0f));
790
791 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
792 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
793 Dragon_Cabeza_RU.RenderModel();
794

```

Resultados:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No hubo problemas

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
- c. Conclusión

Esta práctica tuvo sus complicaciones, debido a que tuve que aprender un poco más sobre Blender para usarlo más eficientemente, en el caso de las animaciones avanzadas resultó fácil comprender como funcionaban, solo se tuvo que pensar bien en que parte se haría la transformación para lograr el efecto deseado.

En el caso del letrero fue más complicado debido a que no sabía si realizar una nueva malla o simplemente reutilizar una, pero opte por generar una nueva debido a que era necesario ajustar los valores de la textura.

En generar se logró cumplir con los objetivos y aprendí nuevas cosas que me pueden ayudar en mi proyecto, específicamente las animaciones avanzadas van a ayudar a que se vea más realista todo lo animado del proyecto.